

Phase 3 — what the signature costs and what it buys

Measurements, not proposals

Abstract

Two kernels answer the same query against the same database. `plain` has never read the generated signature and holds every value as text. `tuned` has read `orders.mli` and holds each column as a dense array of the type the signature names, with a validity mask only where the word `option` appears. Both send identical SQL, receive identical rows, and stream them rather than materialising an intermediate list. The only variable is what the rows become in memory.

The headline is **2.2×**, not the order of magnitude a first estimate suggested, and the sections below say where the rest went. **That is accepted as sufficient**, for the reasons in §6. Every figure is emitted by `dune exec bin/bench.exe` into `docs/phase_3/data/` and read from there by this document, so it can be rebuilt rather than believed.

1 Method, and what is being held constant

The corpus is 200,012 rows of `orders`, one row in three having a NULL `note`. Timings are the median of seven runs; the first is discarded because it pays for the connection and a cold page cache. Variance between whole runs is a few percent, and *no claim here rests on a difference smaller than that* — an earlier draft reported a 14% effect that reversed sign on the next run.

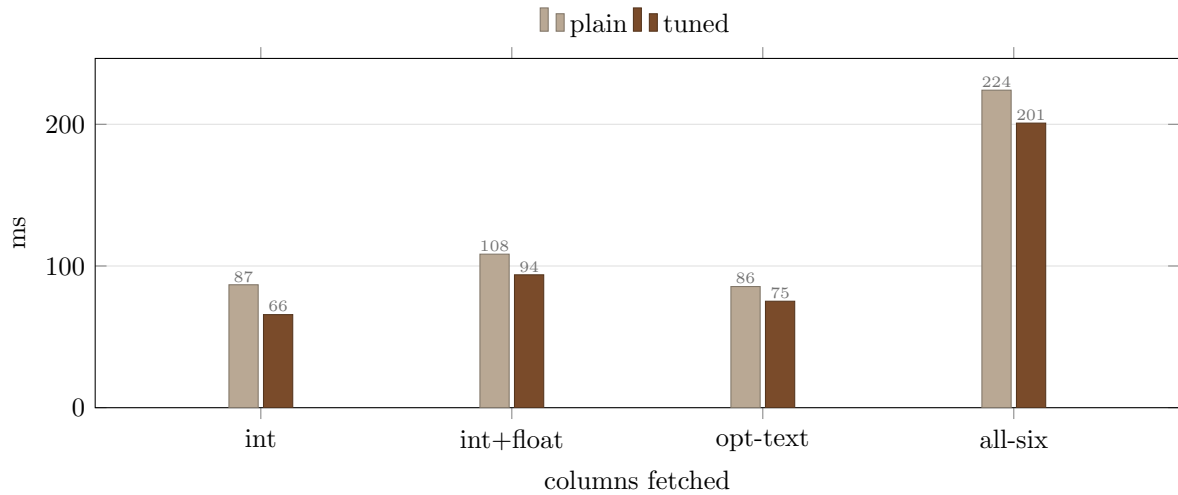
Three things are deliberately held constant, because each is a way of accidentally measuring something other than the signature.

1. **The SQL.** Both kernels push the predicate to Postgres. The database does identical work and the same bytes cross the wire. The only textual difference is that `select *` must stay a star for `plain`, which has no signature telling it what the table contains, while `tuned` expands it.
2. **Streaming.** An earlier version had `tuned` stream while `plain` built an intermediate list, which credited the `.mli` with a 17% saving that belonged to `execute_iter`. Both stream now.
3. **Validation.** `Analysis.plan` rejects a query the signature cannot describe — `qty < 4.5` against an `int` column — before either kernel runs, so neither is answering a different question from the other.

The two kernels are checked for agreement on every measured query. A speed comparison between kernels that return different answers would be worthless, so it is asserted rather than assumed.

2 Materialising a result set

Time to turn one query's rows into each kernel's own answer.

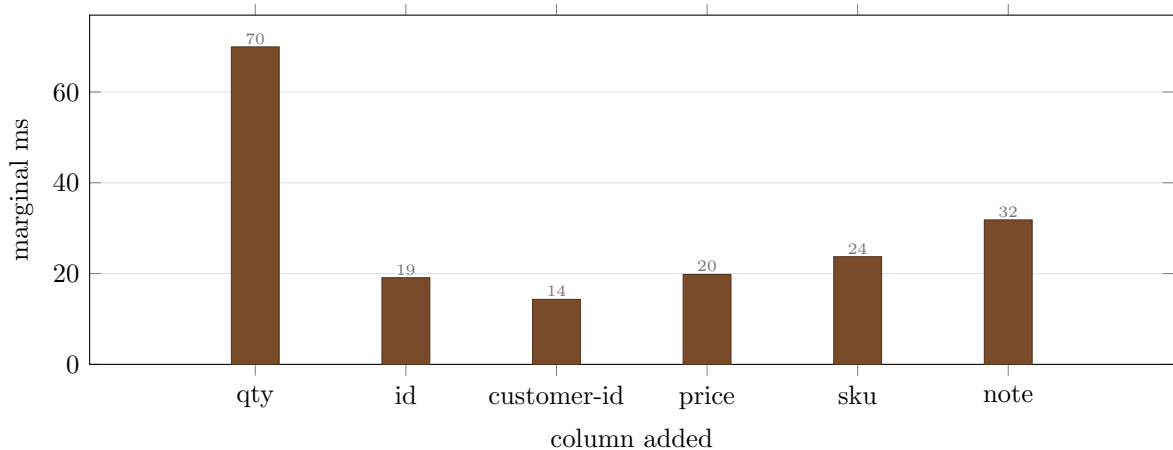


The win tracks the numeric fraction of the columns, and that is the whole explanation. `qty` alone is around 20%: text arrives, and an `int array` is what the signature says to put it in, so the parse happens once instead of being deferred into every later question. A nullable text column is around 10%, and `select *` — four of whose six columns are text or text-like — less still.

This is the first honest disappointment. A string stays a string in both kernels. There is no conversion for the signature to hoist, so on text the `.mli` buys almost nothing at this stage. *A corpus that is mostly strings is a corpus where this claim is weakest*, which is precisely the risk already recorded in `scratch.org` under “what would actually kill this”.

3 What each column costs

Columns added one at a time, so the marginal cost of each is visible.



The first bar carries a fixed per-query cost — the round trip and the row iteration — which is why it is not comparable with the rest. Of the others, the expensive one is `note`, and it is expensive for three reasons that are all legible in its type. `string option` is wider on the wire than an integer, it forces a validity array to be allocated, and it puts a branch in the store path for every row.

This table is predictable from the signature alone, before anything runs.

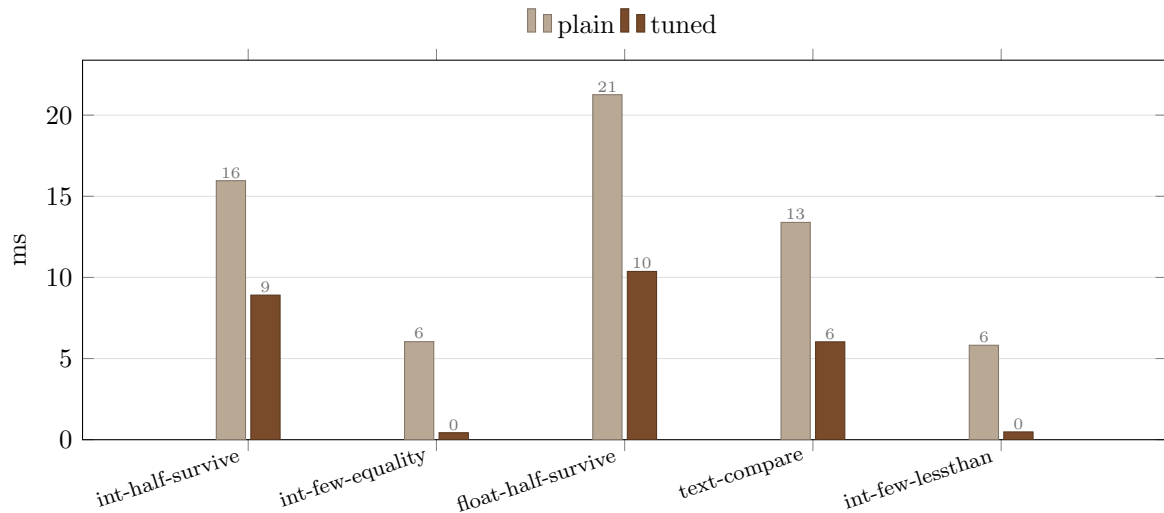
That is the thesis in its literal form: *the access shape is known for free, because we generated the layout*. The signature does not make the fetch faster; it says what the fetch will cost, and

for a placement decision that is the half that matters. It is also the one result here that does not depend on a benchmark at all.

4 Load once, query many

Materialisation alone cannot justify itself: if the answer is wanted once and then discarded, building a dense array and reading it back out is strictly more work than leaving the text alone. The arrays only repay if something runs over them more than once — which is a claim about the workload, not about the compiler.

Each query below runs *in memory*, against a corpus already fetched.

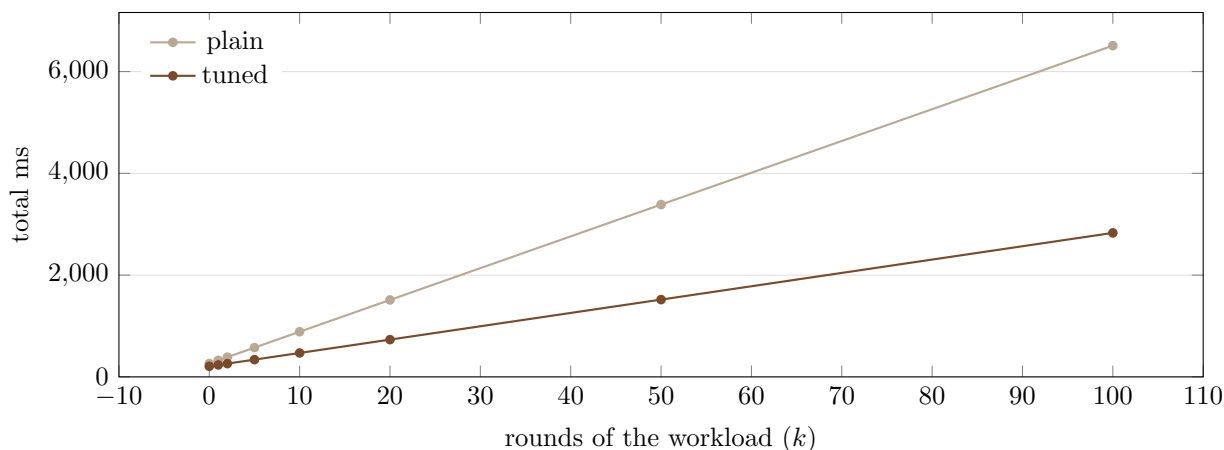


The spread is systematic, and more informative than the average:

- $\approx 13\times$ when the predicate is numeric and few rows survive. **plain** calls `int_of_string` on every row of the corpus; **tuned** compares machine integers in a loop with no validity test, because `qty` is `int` and not `int option`.
- $\approx 2\times$ when the predicate is numeric but half the table survives. The comparison is still $13\times$ cheaper; what closes the gap is materialising the answer.
- $\approx 1.9\times$ on a text predicate — and most of that is **plain** walking a list to reach the column, not anything about types.

5 Where it crosses over

Total time after k rounds of that five-query workload, including the one-off load.



At $k = 0$ the two are level: decoding costs about what **plain** saves by not decoding. The curves separate immediately and flatten towards **2.2** $\times$. The asymptote is not the 13 $\times$ of the scan, because every query still has to build its answer, and that allocation is paid by both.

6 Is 2.2 $\times$ enough

Yes, and the reasons are worth stating rather than leaving to be inferred from the fact that we stopped.

A factor of two is not a constant-factor rounding error. It is the difference between a workload finishing in an afternoon and finishing overnight, and it holds asymptotically rather than at one convenient point on the curve. Nothing about it degrades as the corpus grows, because the plan is not a function of the corpus.

It is also the *floor* of what this arrangement can show, not the ceiling, for three separate reasons.

1. **The thing being computed is as thin as it can be.** One predicate, one projection, no aggregates, no joins. That is the least work it is possible to do over a materialised column and still be doing something, and it is the case least favourable to a typed representation. Every operation the query language gains — a sum, a second predicate, a group-by — runs over arrays that have already been paid for, and lands entirely on the right-hand side of the comparison.
2. **The gap that remains is identified and mechanical.** §7 is not a list of hopes. The survivor `int list` is measurably what collapses 13 $\times$ to 2 $\times$, and a bitmask is a known replacement rather than a research question. We are reporting the number the current implementation earns, not the one it could be made to earn.
3. **The baseline has not been left slow on purpose.** **plain** streams, pushes its predicate down, and is charged nothing for the render that **tuned** pays for. Where it is still carrying avoidable cost — `List.nth` — that is written down rather than banked. A 2.2 $\times$ measured against a baseline that has been given every fair advantage is worth more than a larger figure measured against one that has not.

And the speed was never the load-bearing claim. §3 is: the cost of a column is readable off its type before anything runs, and the plan that follows is a function of the signature and not of the data. A plan that is *merely* twice as fast, arrived at with no cost model, no statistics and no profiling pass, is a stronger result than a faster plan that needed all three — because the former composes with a placement decision and the latter does not. If the number were 1.0 $\times$

the argument would be in trouble. At $2.2\times$ it is not, and chasing $13\times$ before there is anything to place would be optimising the part of the system that is not the question.

7 What is left on the table

Everything below is measured-but-unfixed. None of it is required for the claim; all of it is where the remaining distance between $2.2\times$ and $13\times$ has gone.

The survivor list is the bottleneck.

`Tuned.surviving` returns an `int list` and `gather` walks it, so a query that keeps 100,000 rows allocates 100,000 cons cells and chases them. This, not the comparison, is what collapses $13\times$ to $2\times$ on unselective queries. The replacement is a bitmask — one bit per row in a `Bytes.t`, then a single counting pass to size the output and a second to fill it. Two dense sequential passes instead of a pointer chase, no per-survivor allocation, and the mask is exactly the representation a GPU kernel would produce anyway (§K2 of `kernels.tex`). `plain` cannot follow: it has no dense array to mask.

Fuse the scan with the copy.

With a bitmask, selective queries can skip materialising the intermediate entirely and write survivors straight into the output array as the scan finds them. One pass rather than three.

plain is carrying avoidable cost.

It reaches a column with `List.nth`, which is $O(\text{column index})$ per row. Giving it a positional fast path would shrink the reported advantage, and it should be done before any of the above — a baseline that has been left slow makes the other numbers meaningless.

render is charged to tuned and should not be.

Converting arrays back to text for an HTML table is work `plain` never does, because `plain` never left text. It is a property of the destination, not of the representation. On `select *` it was 208 ms — enough on its own to invert the result — and it is reported as its own phase for that reason.

Data.cell formats floats with `Printf.sprintf "%.2f"`.

Roughly four times the cost of the alternative, and it is inside the render phase above.

Literals are pasted into the SQL text.

`Query.value_to_string` exists to display a query and does not re-escape the quote the lexer already unescaped, so `where sku = 'a''b'` emits `= 'a'b'` — a syntax error, and an injection. The fix is a `pgx` parameter, not a better quoter. Correctness, not performance, and it should be done first.

The plan carries names, not layouts.

`Analysis.filter` and `Analysis.project` hold column names, so `tuned` must re-derive the layout and carries unreachable `failwith` branches for questions the plan has already answered. Holding `Schema.column` instead would make the plan self-sufficient.

8 What this does not show

- **Nothing here is about a GPU.** Both kernels run on one CPU core. What has been established is narrower: that a plan derived from the signature alone is *executable*, and that the representation it dictates is cheaper to query. Whether the same signature can decide device placement is untouched.

- **The workload is assumed, not observed.** Load-once/query-many is the shape that makes shredding worth paying for, and it was chosen for that reason. A workload of single ad-hoc queries would show 1.0×, and the honest reading of §5 is that the claim depends on a property of the user, not of the compiler.
- **One table, one predicate, no joins.** No AND/OR, no aggregates, no ORDER BY, no sub-queries. A join would introduce the dependent traversal that `kernels.tex` argues is the interesting case, and it is absent here entirely.
- **The corpus is synthetic and uniform.** Values are arithmetic rather than random, so a re-seed is reproducible — but selectivity is therefore smooth, and nothing here says what happens on skew.